

第 7 章 指令系统

7.1 机器指令

7.2 操作数类型和操作类型

7.3 寻址方式

7.4 指令格式举例

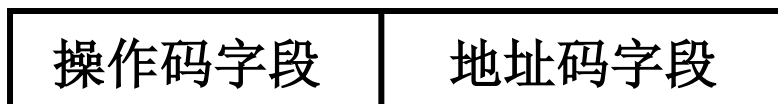
7.5 RISC 技术



7.1 机器指令

- 机器语言：计算机解题直接使用的语言，由一条条语句组成
- 机器指令：每一条机器语言的语句，具有准确的语义
- 指令系统：全部机器指令的集合，集中描述了机器的功能，反映了计算机内部的数据类型、寄存器-存储器结构
- 计算机设计者需要研究如何确定机器的指令系统并实现之
- 计算机使用者依据机器的指令系统，使用汇编语言编制各种程序

一、指令的一般格式



1. 操作码 指出机器所做操作，位数反映种类

(1) 长度固定（集中在指令字的一个字段）

用于指令字长较长的情况 或 RISC 机器

如 IBM 370 操作码 8 位

(2) 长度可变

操作码分散在指令字的不同字段中



- 操作码长度固定的优劣：译码时间短，便于硬件设计；降低了指令位数使用效率（指令操作数个数变化会导致空出一些位，却没有利用），导致指令较长
- 操作码长度可变的优劣：可以有效压缩操作码的平均长度；但增加指令译码、分析的难度，控制器设计复杂
- 采用扩展操作码技术，使得操作码长度随地址数的减少而增加，不同地址数的指令可以具有不同长度的操作码，从而在满足要求的前提下，有效缩短指令字长

(3) 扩展操作码技术

操作码的位数随地址数的减少而增加，以16

位指令为例

OP	A ₁	A ₂	A ₃
----	----------------	----------------	----------------

4 位操作码

0000	A ₁	A ₂	A ₃
0001	A ₁	A ₂	A ₃
⋮	⋮	⋮	⋮
1110	A ₁	A ₂	A ₃

8 位操作码

1111	0000	A ₂	A ₃
1111	0001	A ₂	A ₃
⋮	⋮	⋮	⋮
1111	1110	A ₂	A ₃

12 位操作码

1111	1111	0000	A ₃
1111	1111	0001	A ₃
⋮	⋮	⋮	⋮
1111	1111	1110	A ₃

16 位操作码

1111	1111	1111	0000
1111	1111	1111	0001
⋮	⋮	⋮	⋮
1111	1111	1111	1111

指令字长16位，基本操作码字段4位，另有3个4位长的地址字段，4位基本操作码若全部用于三地址指令，则有16条；若采用扩展操作码技术：

最多15条三地址指令：至少留1条用来扩展

最多15条二地址指令：至少留1条用来扩展

最多15条一地址指令：至少留1条用来扩展

16条零地址指令



(3) 扩展操作码技术

操作码的位数随地址数的减少而增加

	OP	A ₁	A ₂	A ₃
4 位操作码	0000 0001 ⋮ 1110	A ₁ A ₁ ⋮ A ₁	A ₂ A ₂ ⋮ A ₂	A ₃ A ₃ ⋮ A ₃
8 位操作码	1111 1111 ⋮ 1111	0000 0001 ⋮ 1110	A ₂ A ₂ ⋮ A ₂	A ₃ A ₃ ⋮ A ₃
12 位操作码	1111 1111 ⋮ 1111	1111 1111 ⋮ 1111	0000 0001 ⋮ 1110	A ₃ A ₃ ⋮ A ₃
16 位操作码	1111 1111 ⋮ 1111	1111 1111 ⋮ 1111	1111 1111 ⋮ 1111	0000 0001 ⋮ 1111

三地址指令操作码
每减少一种可多构成
2⁴ 种二地址指令

二地址指令操作码
每减少一种可多构成
2⁴ 种一地址指令



例：设指令字长为**16**位，操作数的地址码有**6**位，指令有零地址、一地址、二地址三种格式。

(1) 若**操作码固定**，若零地址指令有**P**种，一地址指令有**Q**种，则二地址指令最多有多少种？

(2) 采用**扩展操作码**技术，若二地址指令有**X**种，零地址指令有**Y**种，则一地址指令最多有几种？

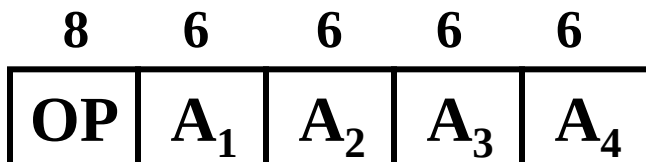
解：(1) 由于操作码固定，所以操作码只有**4**位，能表示**16**种操作，因此结果为： **$16-P-Q$** ；

(2) 二地址指令每减少一种，可扩展 2^6 种一地址指令，一地址指令每减少一种，可扩展 2^6 零地址指令，因此可列出 $((16-X) \times 2^6 - N) \times 2^6 = Y$

因此：
$$N = (16-X) \times 2^6 - Y \times 2^{-6}$$

2. 地址码（指出操作数地址及下条指令地址）

(1) 四地址



A₁ 第一操作数地址

A₂ 第二操作数地址

A₃ 结果的地址

A₄ 下一条指令地址

(A₁) OP (A₂) → A₃

设指令字长为 32 位

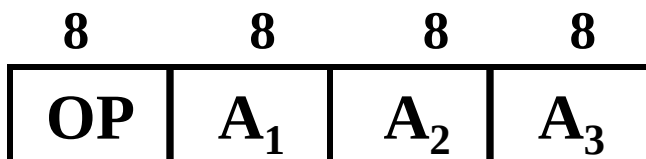
操作码固定为 8 位

4 次访存

寻址范围 $2^6 = 64$

若 PC 代替 A₄

(2) 三地址



(A₁) OP (A₂) → A₃

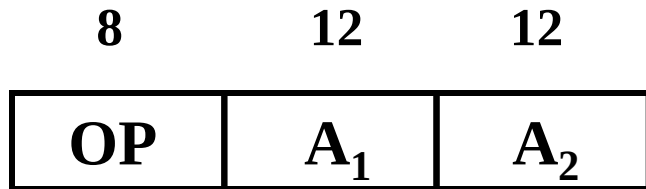
4 次访存

寻址范围 $2^8 = 256$

若 A₃ 用 A₁ 或 A₂ 代替



(3) 二地址



或 (A₁) OP (A₂) → A₁

(A₁) OP (A₂) → A₂

4 次访存

寻址范围 $2^{12} = 4 \text{ K}$

若结果存于 ACC 3次访存

(4) 一地址



若ACC 代替 A₁ (或A₂)

2 次访存

(ACC) OP (A₁) → ACC

寻址范围 $2^{24} = 16 \text{ M}$

(5) 零地址

无地址码或隐含, 如NOP, IRET等指令



- 总结：在指令系统设计时，用PC、ACC等硬件资源承担指令中需要指明的地址码，可以在不改变指令字长的前提下，扩大指令操作数的直接寻址范围，并可以减少对存储器的访问次数
- 其实，地址码部分可以使用通用寄存器作为操作数地址，寄存器访问速度更高

二、指令字长

指令字长决定于 { 操作码的长度
操作数地址的长度
操作数地址的个数

1. 指令字长 **固定** 控制方式简单，早期采用方式
指令字长 = 存储字长 取指令只需一次访存
2. 指令字长 **可变** 控制电路复杂，多字长指令
取指要多次访存

常用指令设计成单字长、短字长指令，以节省存储空间，提高指令运行速度
指令长度按字节的倍数变化



小结

- 当用一些硬件资源代替指令字中的地址码字段后
 - 可扩大指令操作数的寻址范围
 - 可减少访存次数
 - 可缩短指令字长
- 当指令的地址字段全部为寄存器时（如RISC处理器）

三地址 **OP R₁, R₂, R₃**

二地址 **OP R₁, R₂**

一地址 **OP R₁**

- 指令执行阶段不访存
- 可缩短指令字长



7.2 操作数类型和操作种类

一、操作数类型

地址 无符号整数

数字 定点数、浮点数、十进制数

字符 表示文本或字符串；常采用ASCII编码

逻辑数

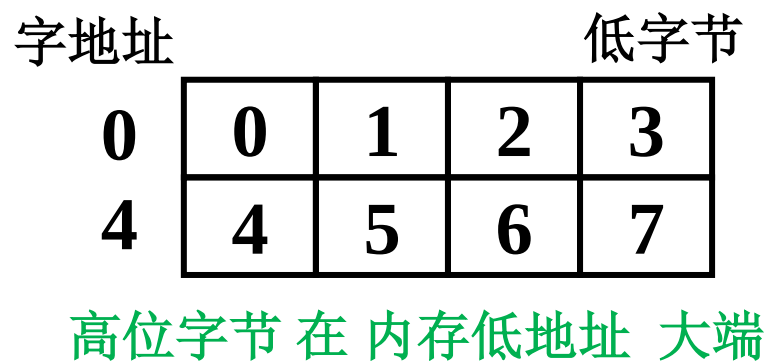
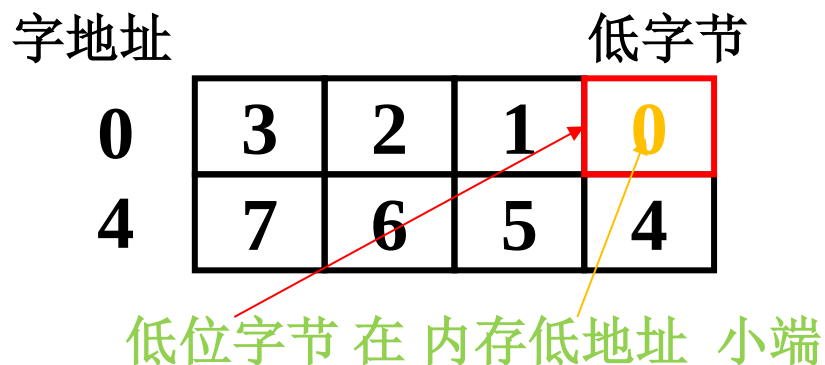
- 多位0和1的组合被看做逻辑数，或者，每一位代表真或假
- 用于逻辑运算 或者 位运算，例如：对多位二进制数据进行位提取、位测试、位设置等位运算



二、数据在存储器中的存放方式

计算机处理的数据的字长（机器字长）各不相同，且是字节的整数倍，**多字节数据在存储器中**存放涉及以下问题：

1. 字节次序 大端、小端模式（第四章有涉及）
2. 边界对准 支持按 字节、字、半字、双字等方式访问，便于硬件实现



为了便于硬件实现，通常要求多字节的数据在存储器的存放方式能满足“边界对准”的要求，在数据不对准的计算机中，一个字可能存储在两个存储单元中，因此需要两次访问存储器，才能取得一个字

例如：计算机存储字长是32位，可按字节、半字、字、双字访问，边界对准是指：字地址是4的整数倍，双字地址是8的整数倍，半字地址是2的整数倍，当所存数据不能满足此要求时，可填充一至多个空白字节

总结：数据存放的起始地址，是数据长度的整数倍

存储器中的数据存放（存储字长为32位）

边界对准

地址（十进制）

字（地址 0）				0
字（地址 4）				4
字节（地址11）	字节（地址10）	字节（地址 9）	字节（地址 8）	8
字节（地址15）	字节（地址14）	字节（地址13）	字节（地址12）	12
半字（地址18）✓		半字（地址16）✓		16
半字（地址22）✓		半字（地址20）✓		20
双字（地址24）▲				24
双字				28
双字（地址32）▲				32
双字				36

边界未对准

地址（十进制）

字(地址2)		半字(地址0)	0
字节(地址7)	字节(地址6)	字(地址4)	4
半字(地址10)		半字(地址8)	8



三、操作类型（面向模型机而不是特定机器）

1. 数据传送

源	寄存器	寄存器	存储器	存储器
目的	寄存器	存储器	寄存器	存储器
例如 汇编指令	MOVE	STORE MOVE PUSH	LOAD MOVE POP	MOVE
置“1”，清“0”				

2. 算术逻辑操作

加、减、乘、除、增 1、减 1、求补、浮点运算、十进制运算
与、或、非、异或、位操作、位测试、位清除、位求反

如 8086 ADD SUB MUL DIV INC DEC CMP NEG
 AND OR NOT XOR TEST



3. 移位操作

算术移位 (有符号数快速*2 /2) 逻辑移位 (无符号数快速*2 /2)

循环移位 (带进位和不带进位)

4. 转移 改变程序执行顺序

(1) 无条件转移 **JMP**

(2) 条件转移 根据指令执行结果决定是否转移，满足条件则转移

结果为零转 ($Z = 1$) **JZ** 如

结果溢出转 ($O = 1$) **JO**

结果有进位转 ($C = 1$) **JC**

跳过一个指令: **SKP**

300

⋮

305

306

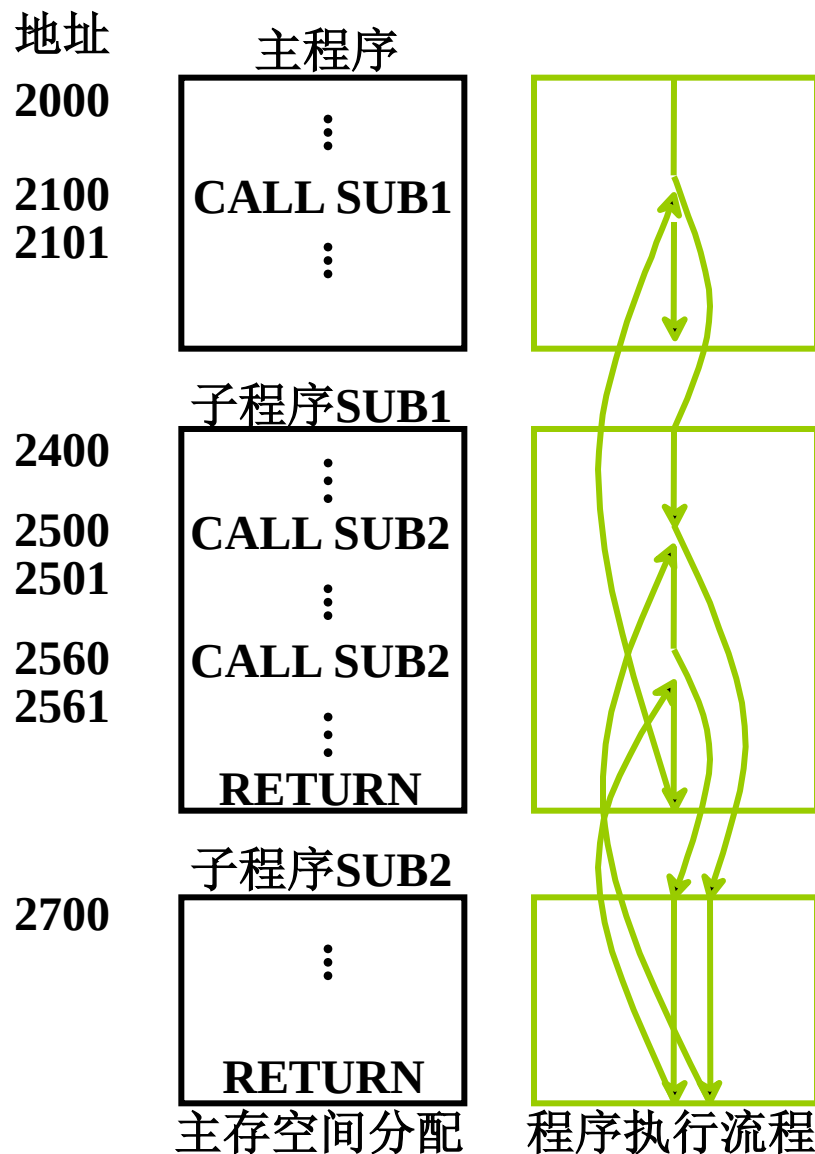
307

完成触发器

SKP DZ $D = 0$ 则跳



(3) 调用和返回



- ✓调用可嵌套
- ✓CALL/RETURN对应使用
- ✓由于子程序可以在多处被调用，必须记住返回地址，可以记在：寄存器、子程序入口地址或者堆栈内

(4) 陷阱 (Trap) 与陷阱指令

陷阱是意外事故的中断，计算机应转入中断程序处理

- 陷阱指令一般不提供给用户直接使用

在出现事故时，由 CPU 自动产生并执行（隐指令）

- 设置供用户使用的陷阱指令

如 8086 INT TYPE 软中断

提供给用户使用的陷阱指令，完成系统调用

5. 输入输出 (对于I/O地址和存储器地址分开的计算机)

输入 端口地址 $\longrightarrow$ CPU 的寄存器

如 **IN AX, n** **IN AX, DX**

输出 CPU 的寄存器 $\longrightarrow$ 端口地址

如 **OUT n, AX** **OUT DX, AX**



7.3 寻址方式

寻址方式 确定 本条指令 的 操作数地址
下一条 欲执行 指令 的 指令地址

寻址方式 { 指令寻址
数据寻址



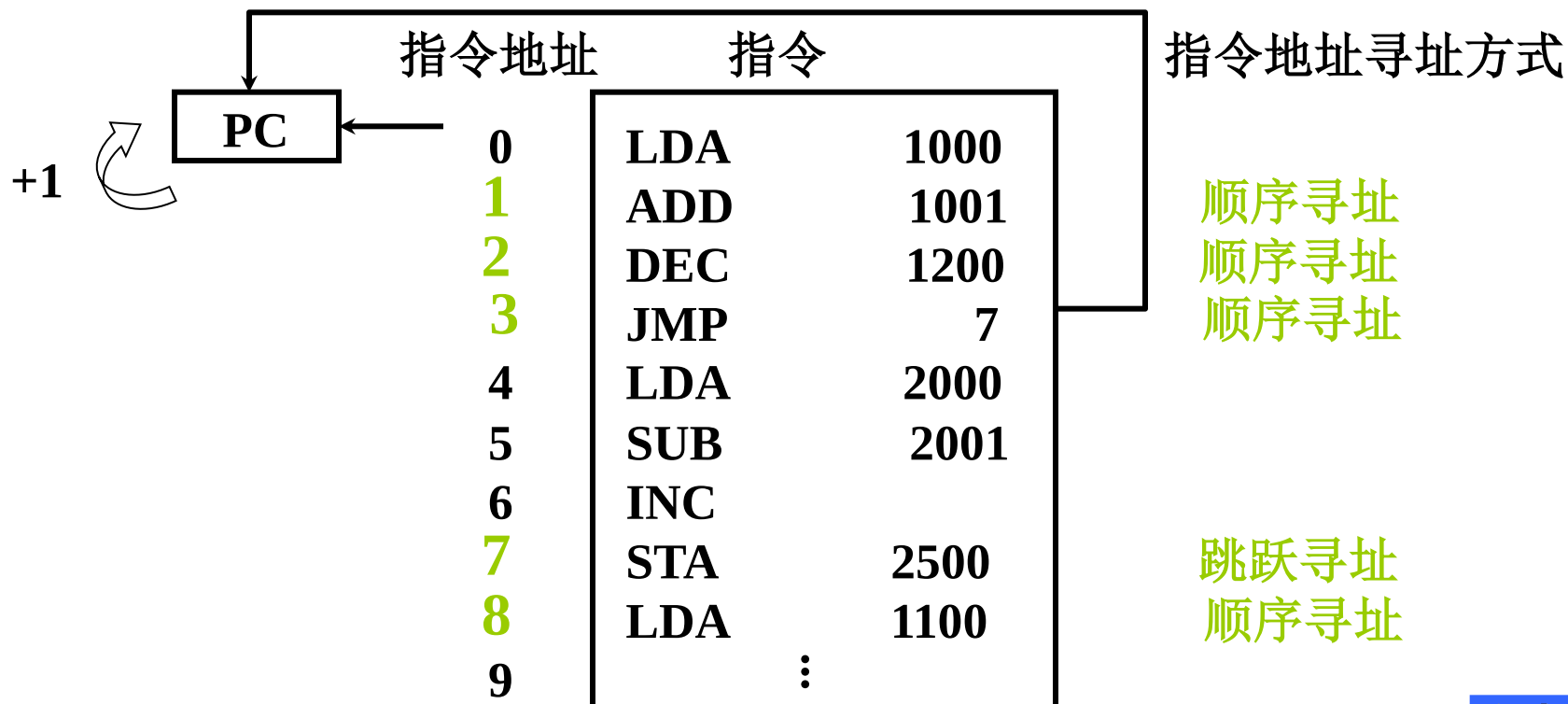
一、指令寻址：分为顺序寻址和跳跃寻址

顺序寻址

$(PC) + 1 \longrightarrow PC$

跳跃寻址

由转移指令指出



二、数据寻址

寻址特征和形式地址构成地址码段

操作码	寻址特征	形式地址 A
-----	------	--------

形式地址 A 指令字中的地址

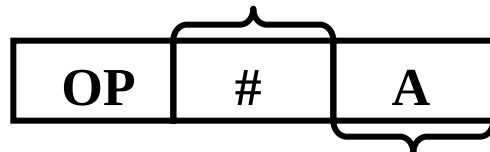
有效地址 EA 操作数的真实地址(在内存中的地址)

约定 指令字长 = 存储字长 = 机器字长

1. 立即寻址

形式地址 A 就是操作数

立即寻址特征



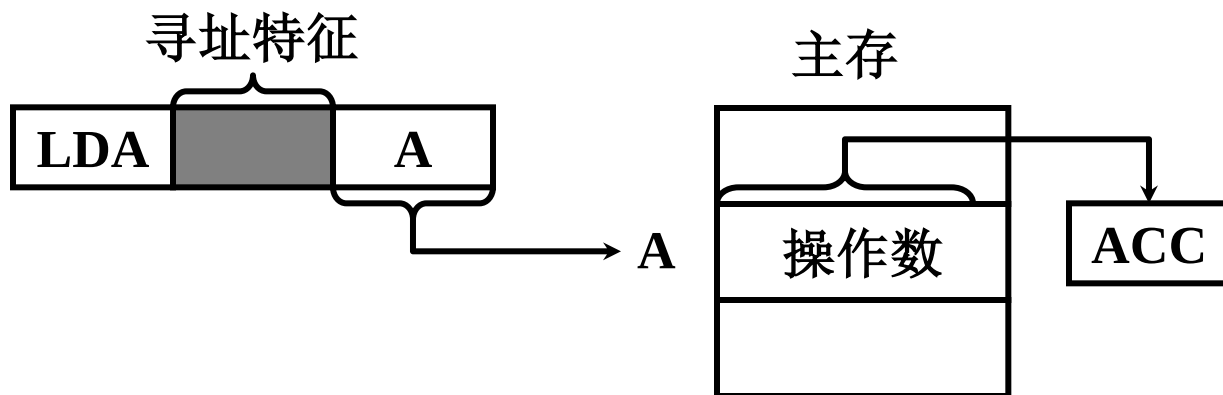
立即数 可正可负 补码

- 指令执行阶段不访存
- A 的位数限制了立即数的范围



2. 直接寻址

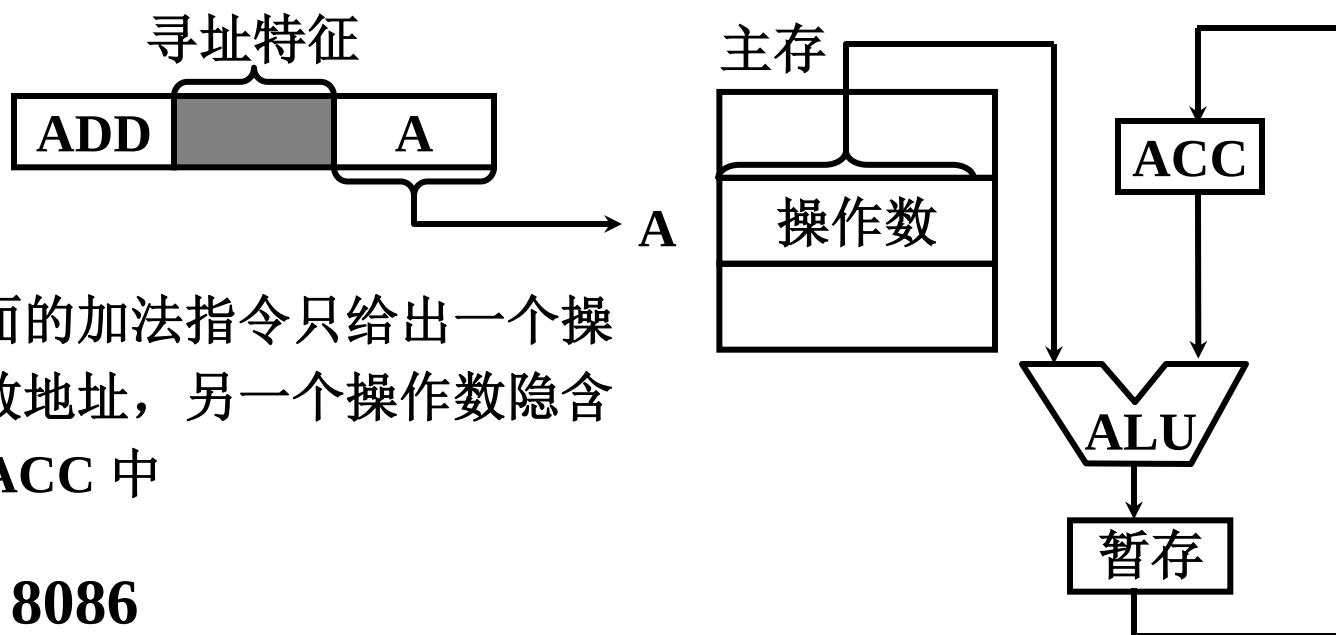
$EA = A$ 有效地址由形式地址直接给出



- 执行阶段访问一次存储器
- A 的位数决定了该指令操作数的寻址范围
- 操作数的地址不易修改（必须修改 A ）

3. 隐含寻址

操作数地址隐含



上面的加法指令只给出一个操作数地址，另一个操作数隐含在 ACC 中

如 8086

MUL 指令 被乘数隐含在 AX（16位）或 AL（8位）中

MOVS 指令 源操作数的地址隐含在 SI 中

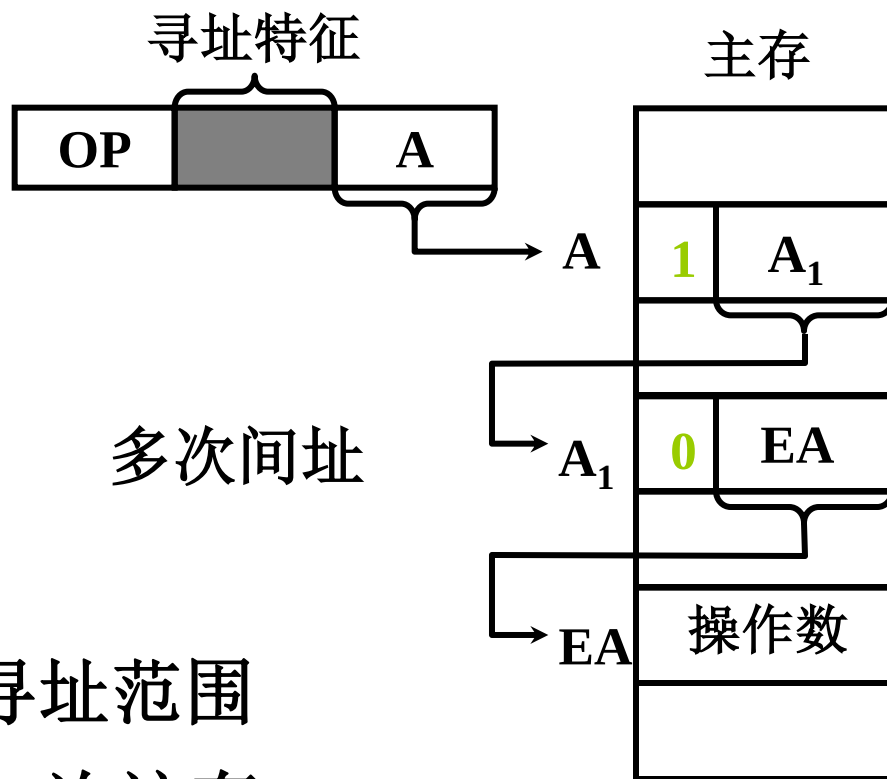
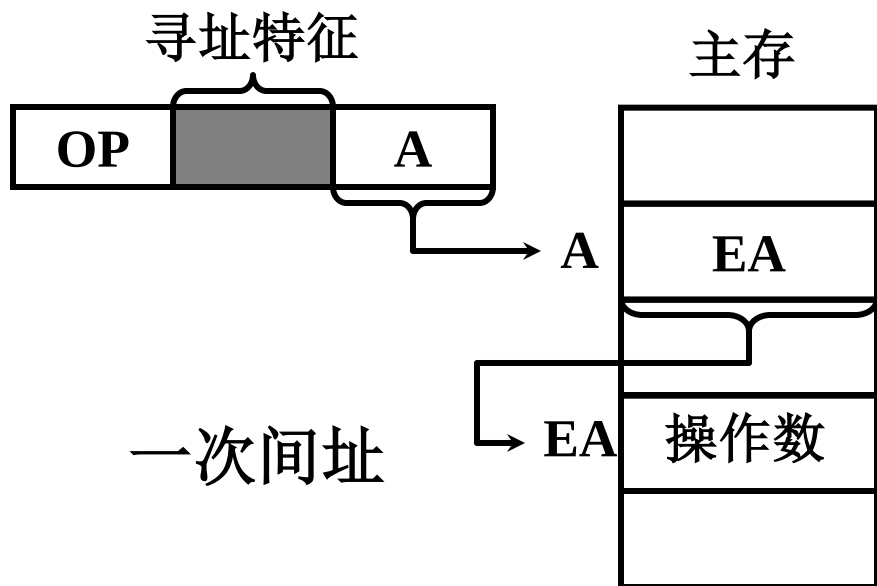
目的操作数的地址隐含在 DI 中

指令字中少了一个地址字段，可缩短指令字长



4. 间接寻址

$EA = (A)$ 有效地址由形式地址间接提供

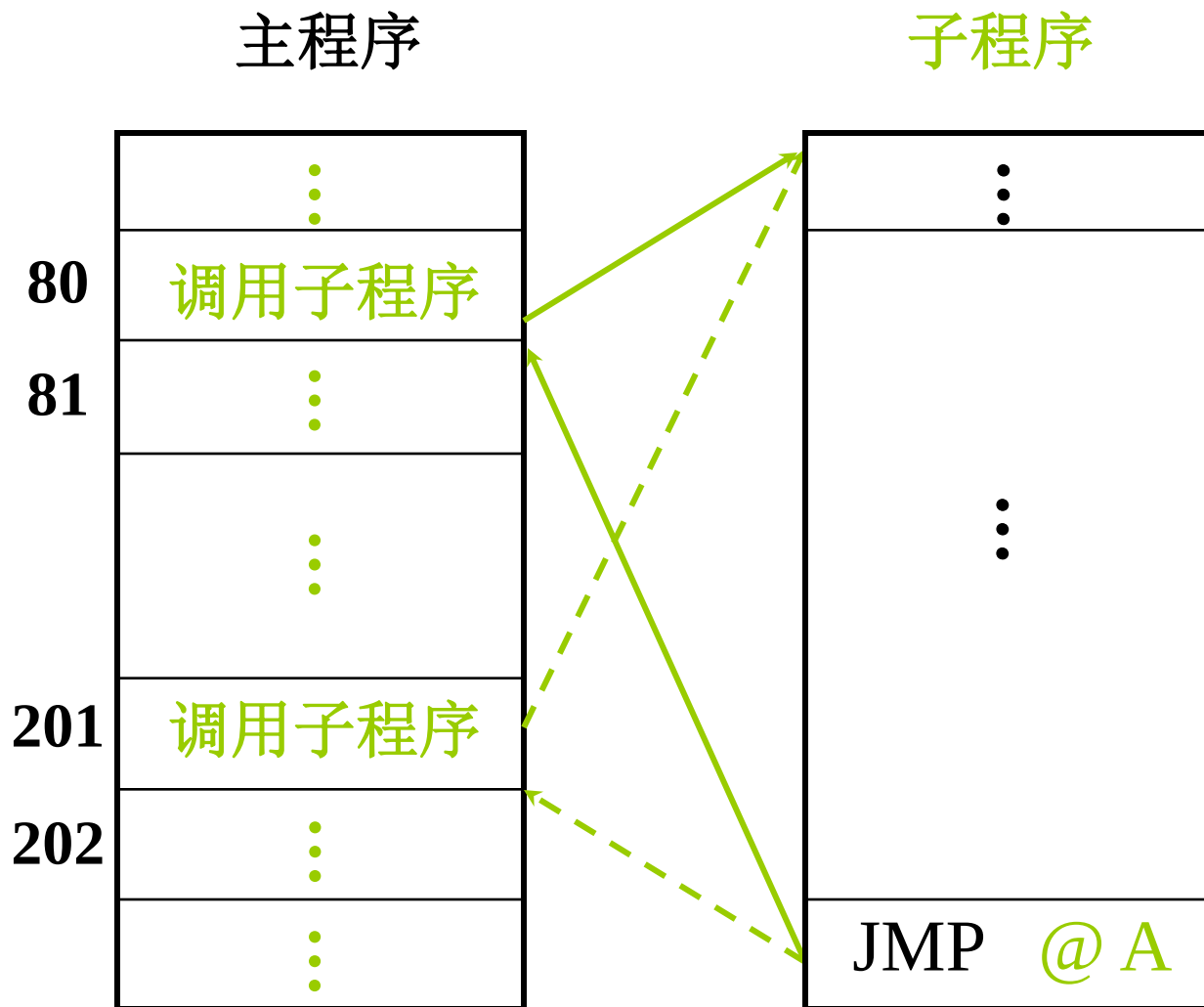


- 优点：可扩大寻址范围
- 执行指令阶段 2 次访存
- 便于编制程序：例如，完成子程序返回

多次访存



间接寻址编程举例



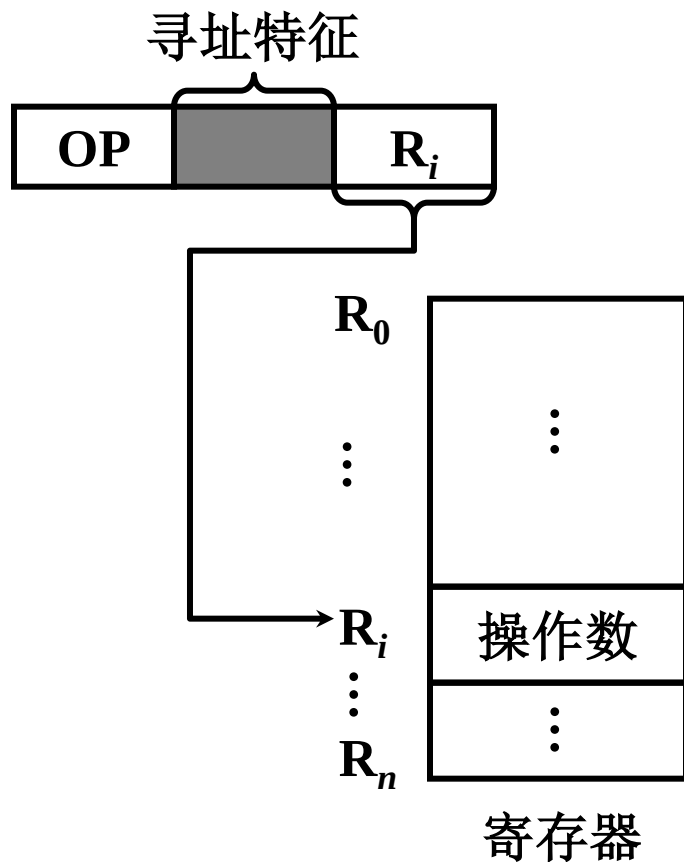
调用子程序前，
先将返回地址
存入子程序最
末条指令的形
式地址 **A** 对
应的存储单元
内，以返回到
原程序断点

@ 间址特征



5. 寄存器寻址

$EA = R_i$ 有效地址即为寄存器编号



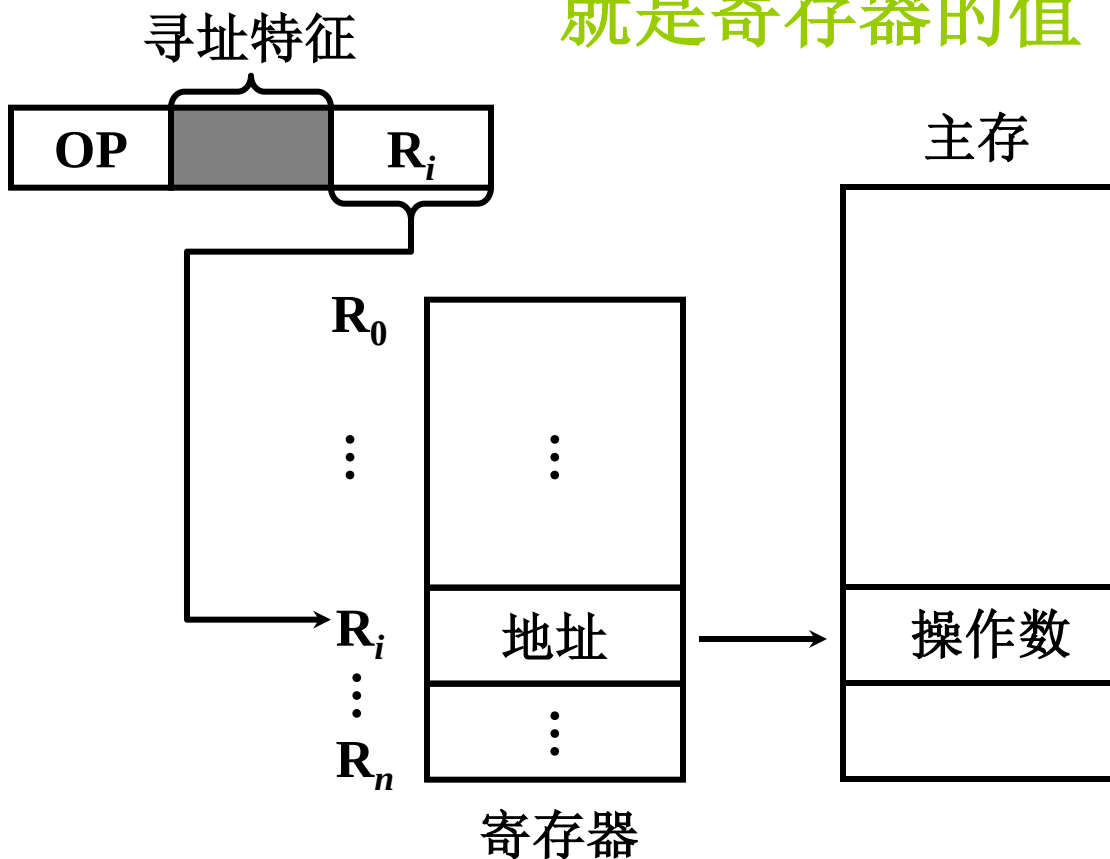
- 执行阶段不访存，只访问寄存器，执行速度快
- 寄存器个数有限，可缩短指令字长



6. 寄存器间接寻址

$$EA = (R_i)$$

有效地址在寄存器中，
就是寄存器的值



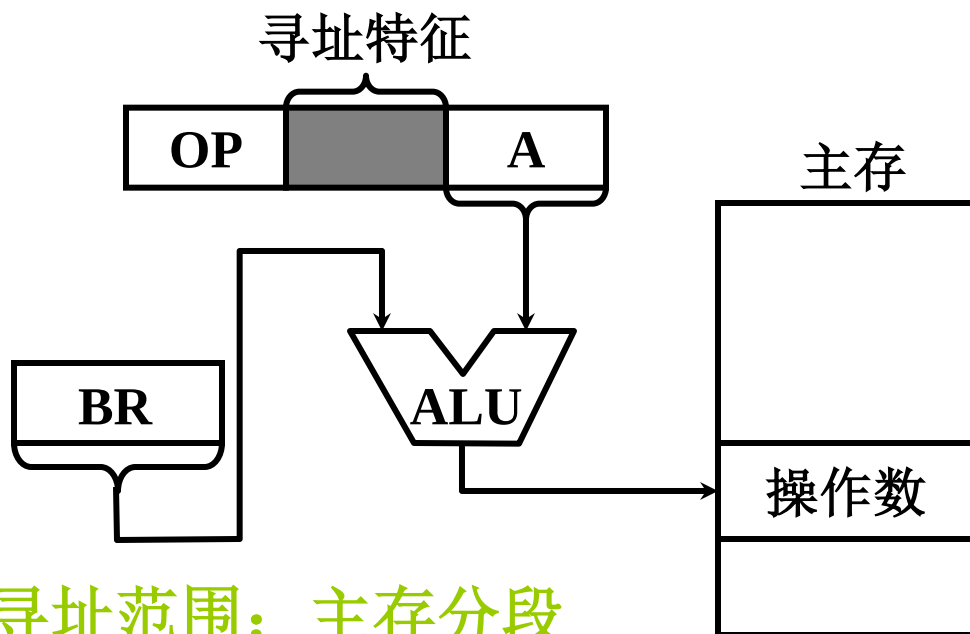
- 有效地址在寄存器中， 操作数在存储器中， 执行阶段访存



7. 基址寻址

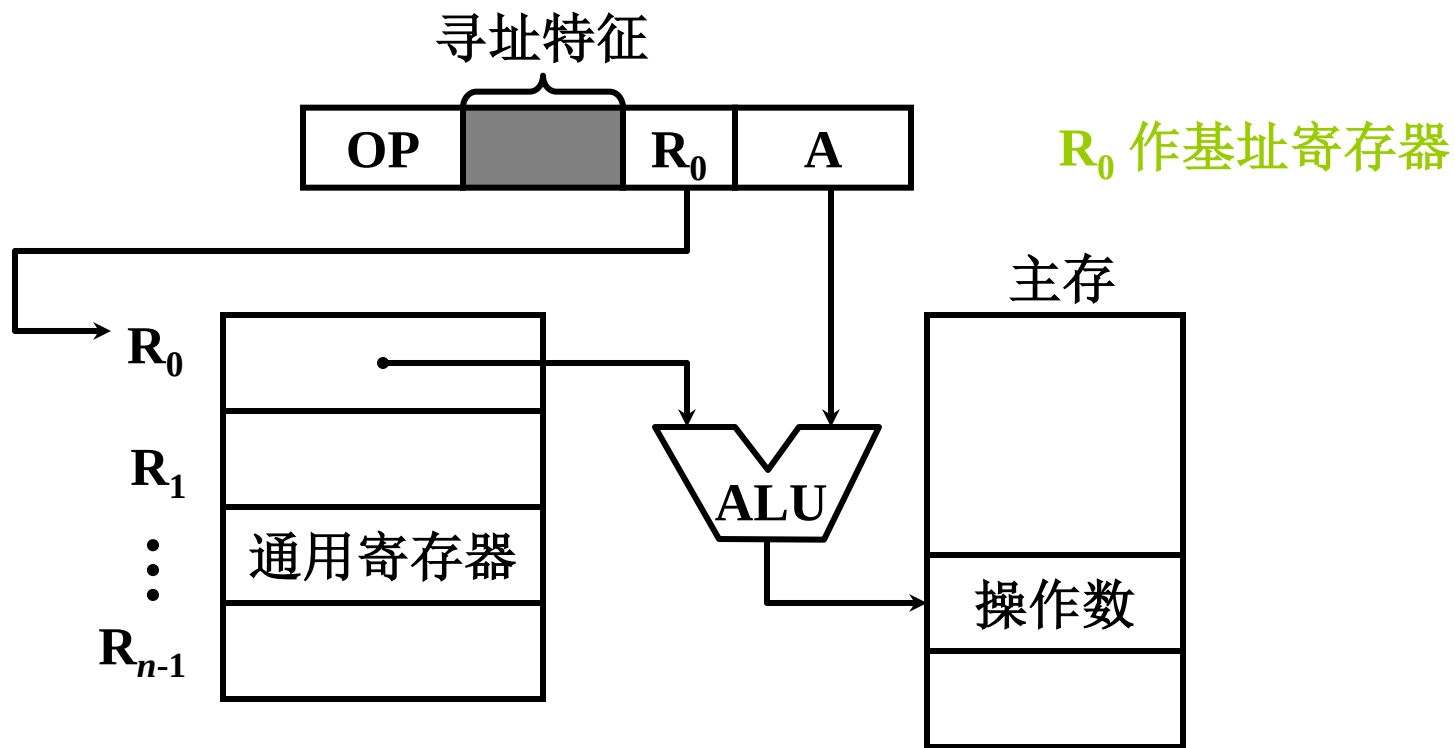
$EA = (BR) + A$ BR 为基址寄存器，可隐式或显式指定

(1) 采用专用寄存器(隐式、编程不指出)作基址寄存器



- 可扩大寻址范围：主存分段
- 有利于多道程序，便于为程序或者数据分配存储空间
 - ✓ BR 内容由操作系统或管理程序确定
 - ✓ 程序执行过程中 BR 内容不变，形式地址 A 可变

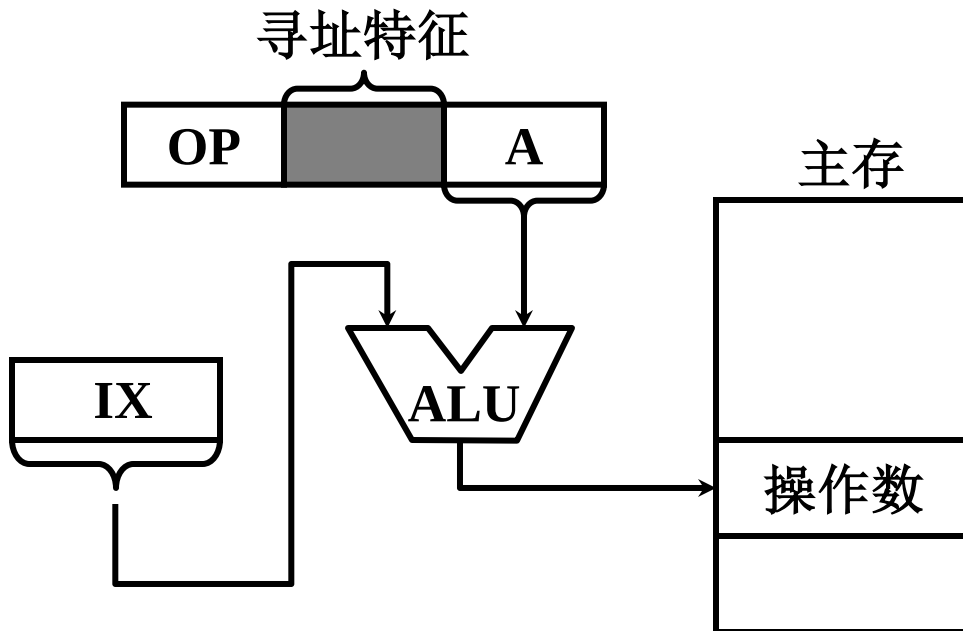
(2) 采用通用寄存器作基址寄存器（用户指定，显式）



- 由用户指定哪个通用寄存器作为基址寄存器
- 基址寄存器的内容可以由操作系统确定
- 在程序的执行过程中 **R₀** 内容不变，形式地址 **A** 可变

8. 变址寻址

$EA = (IX) + A$ IX 为变址寄存器（可专用，隐式给出）
通用寄存器也作为变址寄存器（显式）



- 可扩大操作数寻址范围
- IX 的内容由用户给定
- 在程序的执行过程中 IX 内容可变，形式地址 A 不变
- 便于处理数组问题： A 设成数组首地址

例 设数据块首地址为 D ，求 N 个数的平均值

直接寻址

LDA D

ADD $D + 1$

ADD $D + 2$

⋮

ADD $D + (N - 1)$

DIV $\# N$

STA ANS

共 $N + 2$ 条指令

变址寻址

LDA $\# 0$

LDX $\# 0$ X 为变址寄存器

ADD X, D D 为形式地址

INX $(X) + 1 \rightarrow X$

CPX $\# N$ (X) 和 $\# N$ 比较

BNE M 结果不为零则转

DIV $\# N$

STA ANS

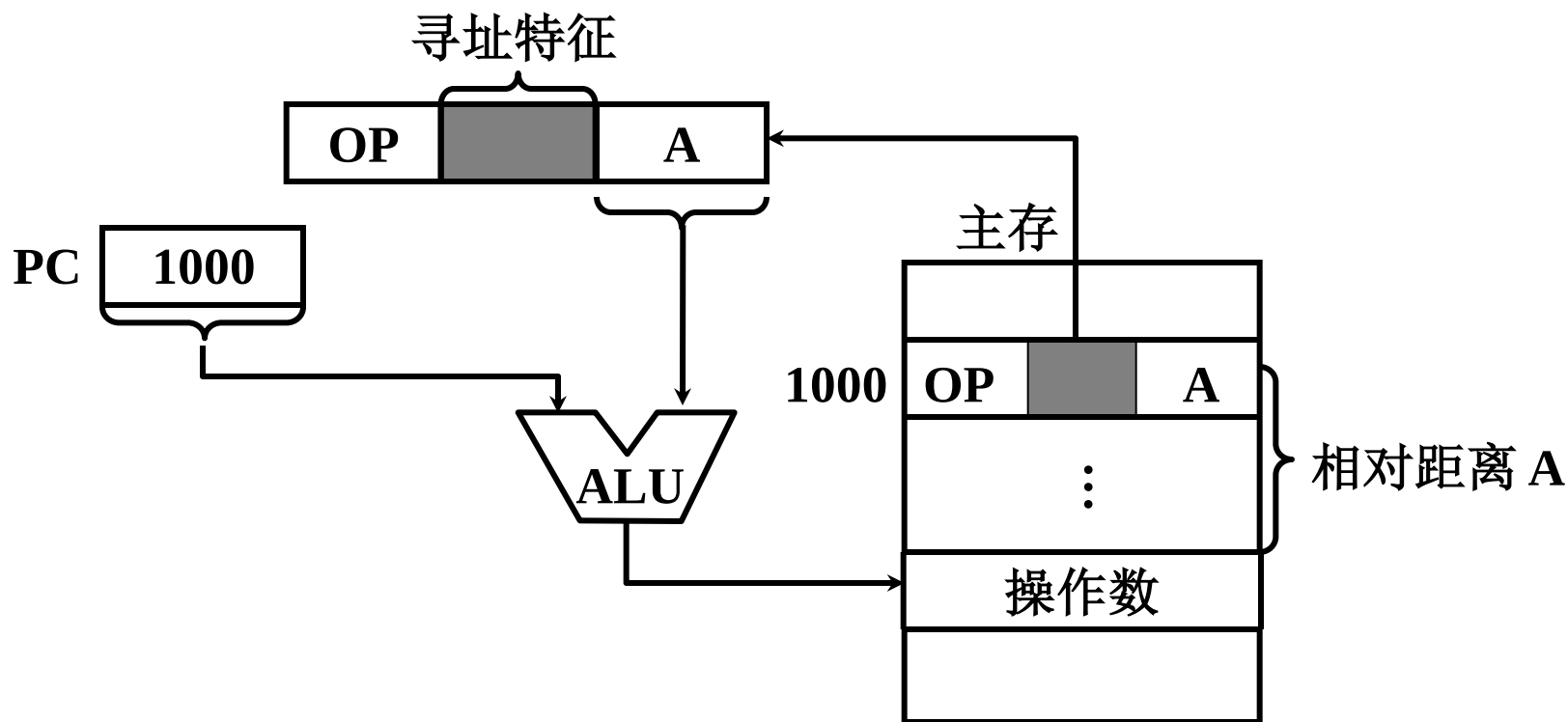
共 8 条指令



9. 相对寻址

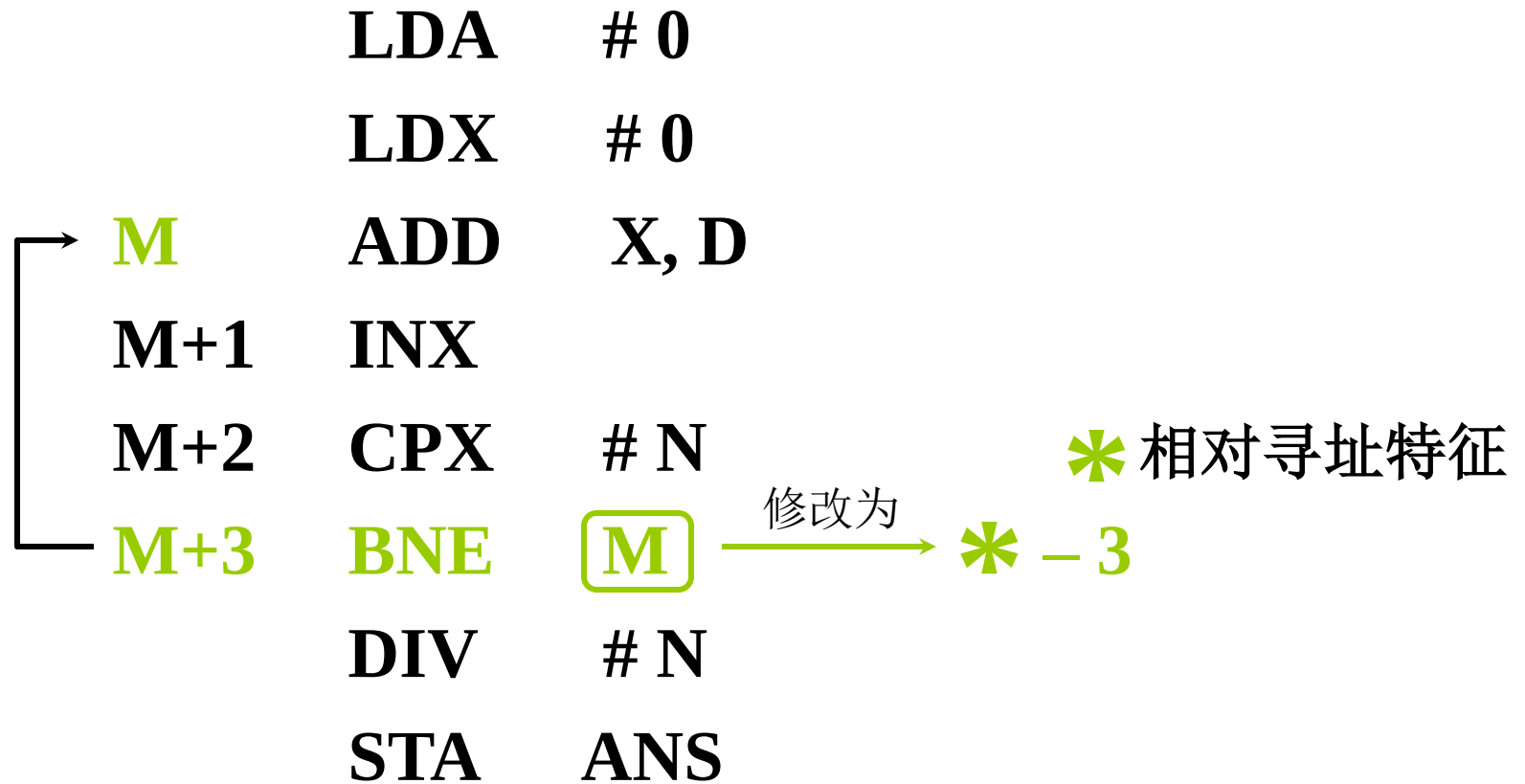
$EA = (PC) + A$ 常用于转移指令的目标地址

A 是相对于当前指令的位移量（可正可负，补码）



- A 的位数决定操作数的寻址范围
- 程序浮动：转移地址随PC值的变化而变化
- 广泛用于转移指令

(1) 相对寻址举例



M 随程序所在存储空间的位置不同而不同

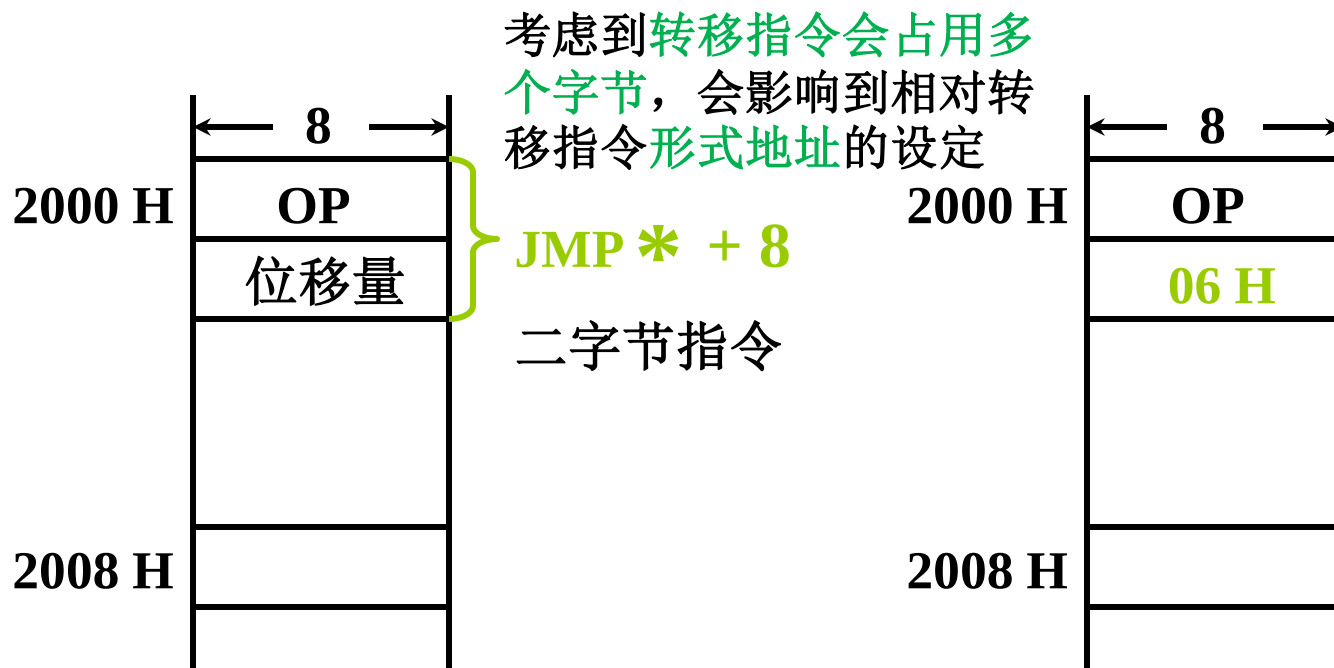
而指令 **BNE * - 3** 与指令 **ADD X, D** 相对位移量不变

指令 **BNE * - 3** 操作数的有效地址为

$$EA = (M+3) - 3 = M$$



(2) 按字节寻址的相对寻址举例



设 当前指令地址 $\text{PC} = 2000\text{H}$

转移后的目的地址为 2008H

因为 取出 $\text{JMP } * + 8$ 后 $\text{PC} = 2002\text{H}$

故 $\text{JMP } * + 8$ 指令 的第二字节为 $2008\text{H} - 2002\text{H} = 06\text{H}$



10. 堆栈寻址 用于保护/恢复中断现场的存储区机制

(1) 堆栈的特点

堆栈 { 硬堆栈
软堆栈

多个寄存器

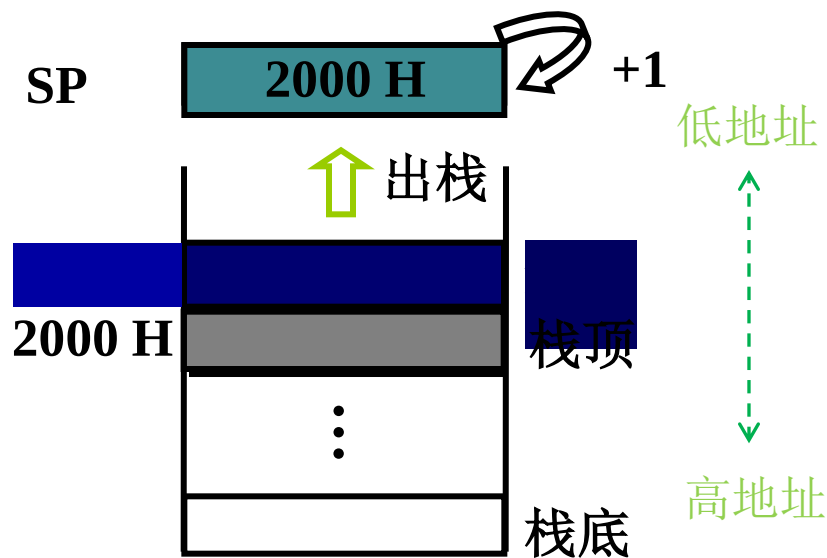
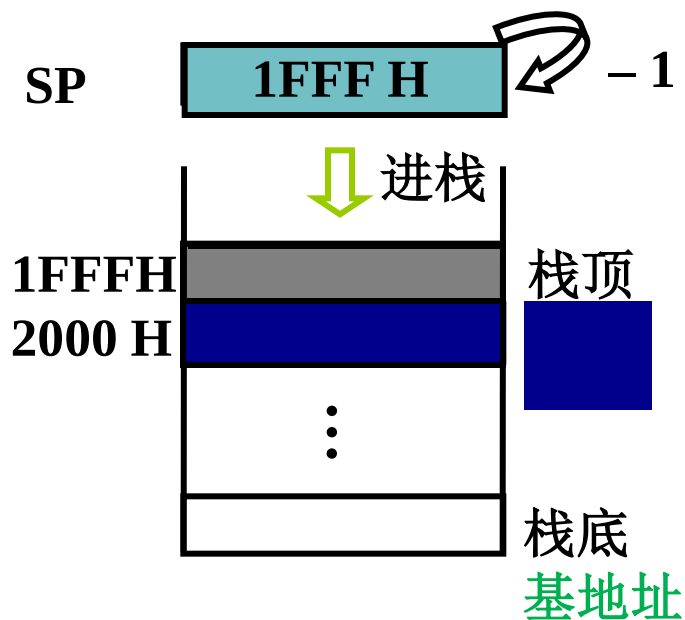
指定的存储空间

操作数只能从栈顶
地址指示的存储单元
存入/取出；数据
存/取都改变SP值

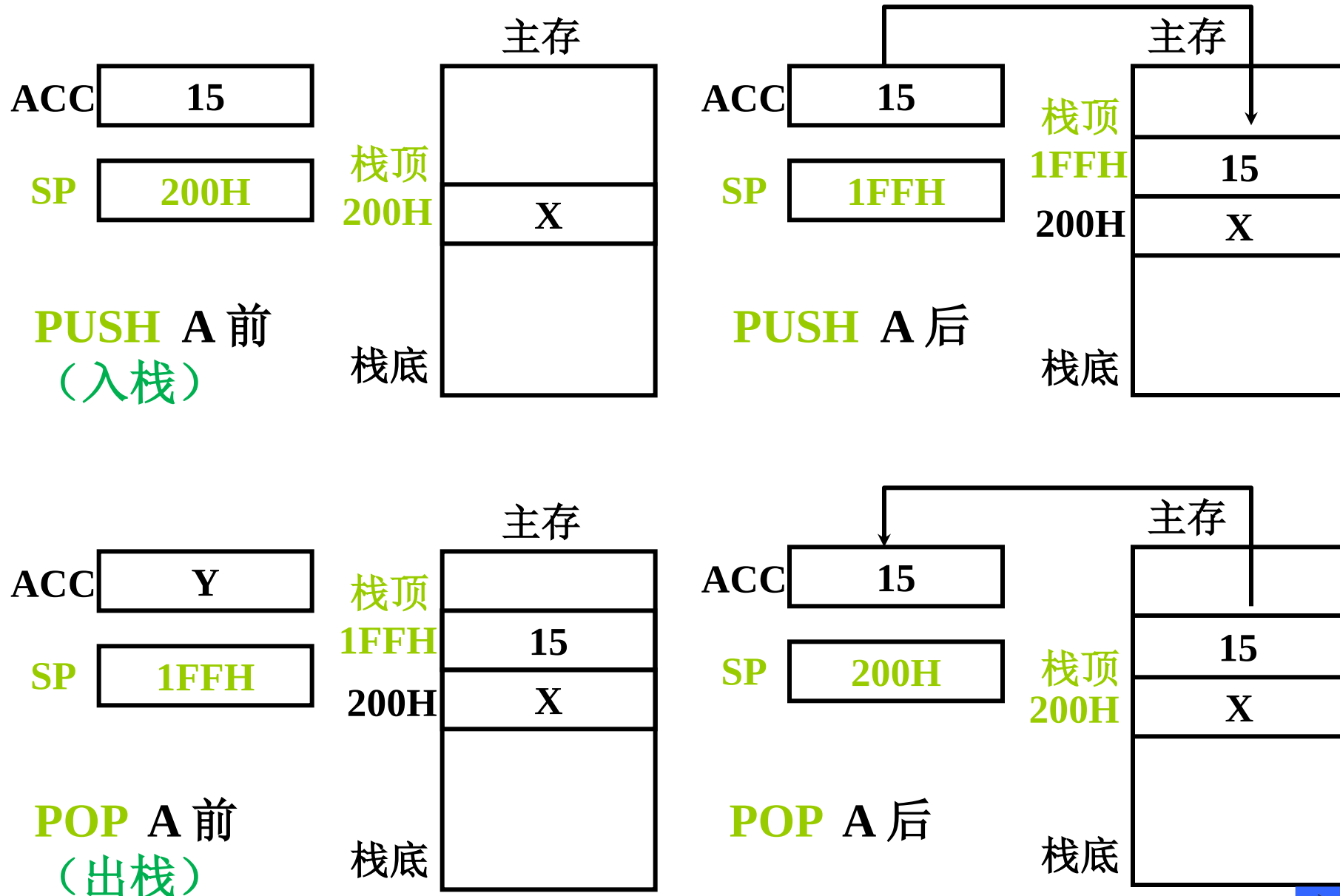
常采用先进后出（一个入出口即栈顶）； 栈顶地址 由 SP寄存器指出

进栈 $(SP) - 1 \rightarrow SP$;

出栈 $(SP) + 1 \rightarrow SP$



(2) 堆栈寻址举例



(3) SP 的修改与主存编址方法有关

① 主存按 字 编址

进栈 $(SP) - 1 \longrightarrow SP$

出栈 $(SP) + 1 \longrightarrow SP$

② 主存按 字节 编址

存储字长 16 位 进栈 $(SP) - 2 \longrightarrow SP$

出栈 $(SP) + 2 \longrightarrow SP$

存储字长 32 位 进栈 $(SP) - 4 \longrightarrow SP$

出栈 $(SP) + 4 \longrightarrow SP$



有关堆栈的说明

- 在任务切换时，堆栈用于保护中间变量、返回地址等；对于**LIFO**方式，数据总是从栈顶加入和取出。
- 随着数据入栈，堆栈有两种增长方向：
 1. 高地址向低地址增长（栈底地址大于栈顶地址）
 2. 低地址向高地址增长（栈顶地址大于栈底地址）

- 满栈与空栈的概念

- ✓ 满堆栈（**full stack**，“**F**”）是指堆栈指针指向堆栈的最后一个已使用的地址或者满位置，也就是堆栈最晚进入的一个数据项位置
- ✓ 空堆栈（**empty stack**，“**E**”）是指**SP**指向堆栈的栈顶上第一个没有使用的地址或者空位置，也就是堆栈最晚进入的数据项的下一个位置
- ✓ 满堆栈的**SP**内容是最后一个已使用的地址，空堆栈**SP**内容是第一个没有使用的地址

例：某机器配有基址寄存器和变址寄存器，采用一地址格式的指令系统，允许直接和间接寻址，且指令字长、机器字长和存储字长都是16位

- (1) 若采用单字长指令，能完成105种操作，则指令可直接寻址的范围是多少，一次间接寻址的范围是多少，画出指令格式并说明各字段的含义。
- (2) 若存储字长不变，则采取什么方法可直接访问容量为16 MB 的主存。

(1)单字长指令为16bits;105种操作---7bits ; 4种寻址方式—2 bits 寻址特征位;地址码为7bits:

OP (7bits)	M(2bits)	AD (7bits)
------------	----------	------------

一次直接寻址 2^7 ，一次间接寻址 2^{16}

(2)16MByte=8M*2Byte=8M存储字= 2^{23} 存储字，可以用23bits地址码寻址所有的存储字，采用双字长指令即可满足要求。

OP (7bits)	M(2bits)	AD1 (7bits)
AD2 (16bits)		

例：某模型机共有64种操作，操作码位数固定，且具有以下特点：

(1)采用一地址或者二地址格式

(2)有寄存器寻址、直接寻址、相对寻址 (-128 ~ +127)三种寻址方式

(3)有16个通用寄存器，算逻运算操作数和结果都在寄存器中

(4)取数/存数指令在通用寄存器和存储器之间传送数据

(5)存储器容量为1MB，按字节编址

要求设计算逻指令、取数/存数指令和相对转移指令的格式，并阐述理由。

解答:

(1) 算术逻辑运算指令: 64种操作—6bits操作码; 寻址方式3种---2bits 寻址特征; 寄存器16个, 因此源操作数寄存器、目的操作数寄存器各4bits, 指令取16位字长

(2)取数/存数指令为寄存器-存储器型, 取双字长32位, 操作码、寻址模式同(1),20bits可寻址1MB存储器

(3)相对转移指令为一地址格式, 取单字长16位, 操作码、寻址模式同(1), 地址码为8bits, 对应位移量是-128~127

例：某计算机主存是 $4M \times 16$ 位，存储字长等于指令字长，若该机指令系统能完成97种操作，操作码位数固定，且具有直接、间接、基址、变址、相对、立即等6种寻址方式

- (1)画出一地址指令格式并指出各字段的作用
- (2)该指令直接寻址的最大范围
- (3)一次间接寻址和多次间址的寻址范围
- (4)立即数范围
- (5)相对寻址的位移量
- (6)上述寻址方式中哪一种执行时间最短，哪一种最长，哪一种便于编制处理数组问题的程序
- (7)如何修改指令，使得指令的寻址范围扩大到4M
- (8)为了能使一条指令转移到主存任何一个位置，可采取何种措施

(1)97种操作，由于 $2^6 < 97 < 2^7$ ，采用7位操作码；6种寻址方式， $2^2 < 6 < 2^3$ 因此采用3位寻址特征位；指令字长等于存储字长为**16**位，因此操作数地址码为**6**位

(2)直接寻址的最大范围是 $2^6 = 64$ 个存储单元

(3)因为存储字长是**16**位，因此一次间接寻址的最大范围是 2^{16} 个存储单元；多次间接寻址时，因为存储字中有一位用来表示是否间接寻址结束，因此寻址范围是 2^{15} 个存储单元

(4)立即数寻址时，采用补码，因为有6位，可表示-32到+31

(5)相对寻址，采用补码方式，6位代码的偏移量是-32到+31

(6)上述方式中，立即数寻址指令执行最快，多次间接寻址方式速度最慢，变址寻址便于解决数组问题

(7)4M空间需要22位地址码，因此可采用双字长指令，即可满足寻址要求；

(8)主存为4M字空间，有两种方法在整个主存空间转移：

1. 采用双字长指令

2. 采用基址寻址：

基址寄存器内容 + 6位形式地址=有效地址

为了得到22位寻址空间，可以把基址寄存器的16位字长的内容左移6位，得到22位字长，再和形式地址相加即可

(* 为相对寻址特征), 就可使该程序浮动至任一地址空间都能正常运行。当执行第 $M+3$ 条指令“BNE * -3”时, 其有效令转至第 M 条指令, 其相对位移量为 -3 , 故当执行第 $M+3$ 条指令“BNE * -3”时, 其有效地址为

$$EA = (PC) + (-3) = M + 3 - 3 = M$$

直接指向了转移后的目标地址。

相对寻址也可与间接寻址配合使用。

例 7.2 设相对寻址的转移指令占 3 个字节, 第一字节为操作码, 第二、三字节为相对位移量(补码表示), 而且数据在存储器中采用以低字节地址为字地址的存放方式。每当 CPU 从存储器取出一个字节时, 即自动完成 $(PC) + 1 \rightarrow PC$ 。

字地址: 地址为偶数的字节: 先取低字节

(1) 若 PC 当前值为 240(十进制), 要求转移到 290(十进制), 则转移指令的第二、三字节的机器代码是什么?

(2) 若 PC 当前值为 240(十进制), 要求转移到 200(十进制), 则转移指令的第二、三字节的机器代码是什么?

解: (1) PC 当前值为 240, 该指令取出后 PC 值为 243, 要求转移到 290, 即相对位移量为 $290 - 243 = 47$, 转换成补码为 2FH。由于数据在存储器中采用以低字节地址为字地址的存放方式, 故该转移指令的第二字节为 2FH, 第三字节为 00H。

先取低字节

2FH
00H

(2) PC 当前值为 240, 该指令取出后 PC 值为 243, 要求转移到 200, 即相对位移量为 $200 - 243 = -43$, 转换成补码为 D5H, 由于数据在存储器中采用以低字节地址为字地址的存放方式, 故该转移指令的第二字节为 D5H, 第三字节为 FFH。

先取低字节

D5H
FFH

10. 堆栈寻址

堆栈寻址要求计算机中设有堆栈。堆栈既可用寄存器组(称为硬堆栈)来实现, 也可利用主存的一部分空间作堆栈(称为软堆栈)。堆栈的运行方式为先进后出或后进先出。

7.5 RISC 技术

一、RISC 的产生和发展

RISC (Reduced Instruction Set Computer)

CISC (Complex Instruction Set Computer)

80 — 20 规律

—— RISC技术

- 典型程序中 **80%** 的语句仅仅使用处理机中 **20%** 的指令
- 执行频度高的简单指令，因复杂指令的存在，执行速度无法提高
- ？ 能否用 **20%** 的简单指令组合不常用的 **80%** 的指令功能



二、RISC 的主要特征

- 选用使用频度较高的一些 简单指令，复杂指令的功能由简单指令来组合
- 指令 长度固定、指令格式种类少、寻址方式少
- 只有 **LOAD / STORE** 指令访存 其余指令操作都在寄存器内完成
- CPU 中有多个 通用 寄存器
- 采用 流水技术 一个时钟周期 内完成一条指令
- 采用 组合逻辑 实现控制器，控制器占芯片面积小，有利于增加其他功能部件，如寄存器
- 采用 优化 的 编译 程序，目标代码体积小



三、CISC 的主要特征

- 系统指令 复杂庞大，各种指令使用频度相差大
- 指令 长度不固定、指令格式种类多、寻址方式多
- 访存 指令 不受限制（数据传送，算逻操作）
- CPU 中设有 专用寄存器
- 大多数指令需要 多个时钟周期 执行完毕
- 采用 微程序 控制器
- 难以 用 优化编译 生成高效的代码



四、RISC和CISC 的比较

1. RISC更能 充分利用 VLSI 芯片的面积

2. RISC 更能 提高计算机运算速度:

译码快: 指令数、指令格式、寻址方式少

执行快: 通用 寄存器多, 访存少

运行快: 控制器采用 组合逻辑, 延迟小

指令周期短: 精简指令系统更适合流水线工作, 多数指令在一个时钟周期内完成

3. RISC 指令系统简单, 便于设计、降低成本提高可靠性

7. RISC 指令少、寻址方式少、通用寄存器多, 有利于编译程序代码优化 效率提高

5. RISC 不易 实现 指令系统兼容



作业

- 7.2;
- 7.15 参照教材例题7.2;
- 7.16

Thank you